

CRA TEAM 3 · CTRL-C

CodeFab

Custom Language를 읽고, 검사하고, 실행하는 Interpreter Factory입니다.

Interpreter Pipeline

Custom Language

Design Patterns

Code Review

GitHub Actions

AGENDA

발표 흐름

1. 팀

조원 역할과 개발 분담

2. 기능

교안 기준 핵심 기능 요약

3. BP

Code Review, TDD,
Refactoring 사례

4. Demo

파일 실행, import, debug,
custom 기능 시연

TEAM

조원 소개와 역할

김명준(팀장)

architecture, infra, CI, coverage, document

권소영

assembler, executor, function, class, import

김민준

expression, statement, shell, file/debug mode

박진우

tokenizer, array, custom language

이예진

checker, optimizer, custom function

GOAL

CodeFab은 작은 Interpreter Factory입니다

입력

사용자가 작성한 `.ctrlc` 소스 코드를 받습니다.

검사

문법 트리를 만들고 실행 전에 오류 가능성을 확인합니다.

실행

검사를 통과한 트리를 순회하며 실제 결과를 출력합니다.

ARCHITECTURE

전체 구조

Tokenizer

문자열을 Token 목록으로 분해

Assembler

Token을 Expr/Stmt 트리로 조립

Checker

scope, 호출, 타입 검사

Executor

AST 순회, 출력, 상태 변경

FEATURE SCOPE

교안 기준 구현 범위

영역	대표 기능	발표 초점
기본 문법	변수, 출력, 연산, 조건문, 반복문	언어가 실행되는 최소 흐름
구조화	Func, Class, This, Super, instanceof	함수와 객체 모델
자료 구조	Array(size), index get/set	런타임 오류와 checker 연결
확장	import, alias, custom function	언어 확장 방식
운용	prompt, file, debug shell	Interpreter Factory 제어

기본 문법

기능 요약

변수, 출력, 연산, 조건, 반복으로 프로그램의 기본 흐름을 표현합니다.

구현 방법

Token을 statement/expression으로 조립하고 Executor가 순서대로 평가합니다.

```
var total = 0;

for (var i = 0; i < 3; i = i + 1) {
    total = total + i;
}

if (total > 2) {
    print total;
}
```

FEATURE · FUNCTION

함수

기능 요약

Func 로 코드를 묶고, parameter와 return으로 값을 주고받습니다.

구현 방법

함수 선언을 환경에 저장하고 호출 시 새 scope에서 body를 실행합니다.

```
Func add(a, b) {  
    return a + b;  
}  
  
print add(2, 3);
```


FEATURE · CLASS

클래스와 상속

기능 요약

instance 생성, method 호출, `This`, `Super`, `instanceof` 를 지원합니다.

패턴 포인트

Class 생성도 함수 호출처럼 다뤄 Adapter/Callable 구조와 연결됩니다.

```
Class Robot {  
    move() { print "move"; }  
}  
  
Class SpeedRobot : Robot {  
    move() {  
        Super.move();  
        print "speed";  
    }  
}  
  
print SpeedRobot() instanceof Robot;
```

FEATURE · ARRAY

정적 배열

기능 요약

`Array(size)` 로 고정 크기 배열을 만들고 index로 읽고 씁니다.

구현 방법

Executor는 index 범위와 타입을 검사하고, 잘못된 접근은 runtime error로 보고합니다.

```
var arr = Array(3);  
arr[0] = 10;  
arr[1] = 20;  
  
print arr[0];  
print arr[1];
```

FEATURE · IMPORT

Library Import

기능 요약

다른 `.ctrlc` 파일의 선언을 alias namespace로 가져옵니다.

구현 방법

import 대상은 선언 중심으로 읽고, alias 충돌과 순환 import를 검사합니다.

```
// main.ctrlc
import "sum.ctrlc" alias sum;

print sum.add(1, 2);
```

```
// sum.ctrlc
Func add(a, b) {
    return a + b;
}
```

FEATURE · CHECKER

실행 전 오류 검증

Scope

중복 선언, 미초기화 변수, alias 충돌을 실행 전에 잡습니다.

Function/Class

잘못된 return, This/Super 위치, 자기 상속 같은 오류를 검사합니다.

Import

반복문 내부 import, 같은 scope 중복 import, 순환 import를 막습니다.

FEATURE · OPTIMIZATION

정적 바인딩과 실행 전 최적화

기능 요약

Checker가 변수까지의 scope distance를 미리 계산해
Executor lookup을 줄입니다.

구현 방법

실행 전에 binding 정보를 저장하고, literal-only expression
은 미리 접습니다.

```
{  
  var a = 10;  
  {  
    print a;  
  }  
}
```

```
// before: parent scope를 순차 탐색  
// after: 미리 계산한 distance로 접근
```

FEATURE · SHELL

Interpreter Factory 제어

Prompt

한 줄 입력마다 pipeline을 실행합니다.

File

`make run sample.ctrlc` 로 파일 전체를 실행합니다.

Debug

breakpoint, step, watch로 실행 상태를 관찰합니다.

CUSTOM EXTENSION

팀 색깔을 담은 확장 기능

한글 alias

영어 키워드와 같은 의미를 한글 표현으로도 작성합니다.

`ctrl_c()`

CodeFab 코드 안에서 팀명, 의미, 역할을 소개하는 custom function입니다.

♡ 연산

두 문자열 이름의 궁합 점수를 계산하는 발표용 custom 연산입니다.

DESIGN PATTERN MAP

좋은 패턴이 보이는 지점

패턴	적용 지점	효과
Composite	Expr/Stmt AST	부분 expression과 전체 statement를 같은 방식으로 순회
Adapter	Callable 호출 규칙	내장 함수, 사용자 함수, class 생성을 같은 call 흐름으로 처리
Factory-like	Tokenizer/Assembler	token과 AST node 생성 책임을 한 곳에 모음
Command	Shell command	shell 명령을 독립 실행 단위로 분리해 적용

AST를 트리로 다루기

문제

표현식과 문장은 중첩될 수 있어 단순 list만으로 실행하기 어렵습니다.

해결

`LiteralExpr`, `BinaryExpr`, `PrintStmt` 같은 node를 공통 트리로 구성합니다.

```
print 1 + 2 * 3;
```

```
PrintStmt
```

```
└ BinaryExpr(+)
```

```
  └ LiteralExpr(1)
```

```
  └ BinaryExpr(*)
```

```
    └ LiteralExpr(2)
```

```
    └ LiteralExpr(3)
```

서로 다른 호출 대상을 같은 규칙으로

문제

내장 함수, 사용자 함수, class 생성자는 내부 구현이 서로 다릅니다.

해결

공통 `call()` 규칙을 두어 Executor가 호출 대상의 세부 구현을 몰라도 됩니다.

```
clock();           // NativeFunction  
add(1, 2);         // UserFunction  
Robot("A");       // UserClass
```

```
Executor -> callee.call(arguments)
```

PATTERN · COMMAND

Shell 명령에 Command Pattern 적용

Before

```
if source == "ctrlc":  
    recommend_command()  
if source == "ctrlv":  
    run_recommended_command()  
if source.startswith("explain "):  
    explain_source(code)
```

After

```
RecommendCommand.execute(shell)  
RerunCommand.execute(shell)  
ExplainCommand.execute(shell)  
RunCodeCommand.execute(shell)  
  
ShellCommandFactory.create(source)  
PromptShell -> command.execute(shell)
```

Code Review 사례

발표 포인트

- PR template로 변경 의도와 검증 범위를 먼저 공유했습니다.
- 최소 2명 approve 기준으로 merge 전 관점을 늘렸습니다.
- 리뷰를 통해 debug 출력, import 경로, checker 누락을 보강했습니다.

TDD와 회귀 테스트 사례

발표 포인트

- 기능 구현 전에 실패 케이스를 테스트로 고정했습니다.
- import, debug, checker 오류를 반복 가능한 테스트로 남겼습니다.
- GitHub Actions가 PR마다 같은 기준으로 검증했습니다.

Refactoring 사례

발표 포인트

- Shell command를 Command Pattern으로 분리했습니다.
- 기능 추가 위치가 명확해지고 테스트 단위가 작아졌습니다.
- 기존 패턴 적용 사례와 함께 설계 개선 흐름을 설명합니다.

GITHUB COLLABORATION

협업 규칙

PR Rule

- PR template 작성
- 최소 2명 approve
- 작성자가 직접 merge

Actions

- Lint
- Unit Tests
- Gist Acceptance

Protection

- main 직접 반영 방지
- 리뷰 후 merge
- check 통과 확인

LABELS

PR 성격을 label로 구분했습니다

feature

bug

test

refactor

documentation

chore

효과

리뷰어가 PR의 목적을 먼저 파악하고, 어떤 관점으로 봐야 하는지 빠르게 정할 수 있습니다.

발표 연결

기능, 버그 수정, 테스트, 리팩터링 사례를 label 기준으로 나누어 설명할 수 있습니다.

DEMO

시연 순서

1

`make run sample.ctrlc`
로 기본 실행

2

import로 외부 선언 사용

3

debug mode에서 breakpoint
와 watch 확인

4

`ctrl_c()` 와 custom 연산 시
연

RETROSPECTIVE

소감

기능보다 구조

기능이 늘어날수록 pipeline과 책임 분리가 중요해졌습니다.

리뷰보다 대화

리뷰는 승인 절차가 아니라 서로의 구현 의도를 맞추는 과정이었습니다.

테스트보다 신뢰

테스트가 남아 있어 다음 변경을 더 자신 있게 할 수 있었습니다.